

# BINV2150-A – Javascript : avancé

## Séance 02 – Le langage TypeScript

Jérome Plumat & Sébastien Strebelle

Haut École Léonard de Vinci

2025-2026



Haute École
Léonard
de Vinci

# Introduction



Le langage TypeScript (TS) est un langage de programmation remplaçant progressivement le langage JavaScript (JS). Ce lanage sera utilisé pour développer les applications web,

- Côté client (dans navigateur, à l'aide de la librairie React),
- Côté serveur (dans serveur, à l'aide de Node.js et Express).

Le langage TypeScript est un langage de programmation qui n'est pas directement exécutable. Il est nécessaire de le compiler (transpiler) en JavaScript avant de pouvoir l'exécuter.

# TypeScript : les variables



```
let a = 1;              // variable with type inference
const b = 2;            // constante
b = 3;                  // error
const c : number = 3; // variable with type annotation
```

# TypeScript : les types



Il existe deux façons de typer une variable :

- l'annotation de type,
  Le type est déclaré explicitement.
- l'inférence de type,
  Le type est déterminé automatiquement par le compilateur.

## Attention

Il est obligatoire d'annoter explicitement le type :

- pour les variables dont le type n'est pas un type de base,
- pour les paramètres des fonctions,
- pour les retours des fonctions.

# TypeScript : les fonctions (1)



Il existe (au moins) deux façons de déclarer une fonction :

- la déclaration classique,
- la déclaration de fonction fléchée.

```typescript
// Classical function declaration
function add1(a: number, b: number): number {
  return a + b;
}
// Arrow function
const add2 = (a: number, b: number): number => a + b;
// calling the functions
add1(1, 2);
add2(1, 2);
```

# TypeScript : les fonctions (2)



On utilisera les déclaration classique dans les déclarations de `classe`.

```
Class MyService {
  public static getAll( filter : SomeFilter) : myData[ ] {
    // ...
  }
}
```

Dans ce cas, une fonction est déclarée avec le pattern :

`<visibilité> <nom>(<paramètres>) : <typeRetour> {corps}`

# TypeScript : les fonctions (3)



On utilisera les déclaration de fonction fléchée dans principalement deux cas :

- pour la stocker dans une variable,
- pour la passer comme paramètre d'une fonction.
  Insistons ici : **on passe une fonction en paramètre d'une autre fonction**.

```
const myFunction = (param1: type1, param2: type2): returnType => {
  // ...
};

someOtherFunction((param1: type1, param2: type2): returnType => {
  // ...
});
```

Le template est le suivant :

`(<paramètres>) : <typeRetour> => {corps}`

# TypeScript : les fonctions (2bis)



Nous l'avons déjà rencontrée l'année passée dans le cours de web1.

```
app.get('/user/:id', (req: Request, res: Response) => {
  // ...
});
```

Ici, on passe à la fonction `get` de l'objet `app` une fonction. Cette fonction a elle même deux paramètres :

- `req` de type `Request`,
- `res` de type `Response`.

# TypeScript : les objets



En TS, un objet est une collection de propriétés et de méthodes. On peut le créer explicitement :

```
const myObject = {
  property1: value1,
  method1: (param1: type1, param2: type2): returnType => {
    // ...
  }
};
```

On peut aussi créer un objet en instanciant une classe.

```
class MyObject {
  property1: value1;
  method1(param1: type1, param2: type2): returnType {
    // ...
  }
}
const myObject = new MyObject();
```

# TypeScript : les tableaux (1)



Un tableau est une collection d'objets de même type.

```
const myArray: type[ ] = [value1, value2, value3];
myArray.push(value4);
myArray.pop();
const length = myArray.length;
```

# TypeScript : les tableaux (2)



Les tableaux étant des objets on peut appeler des méthodes sur eux. Et on peut passer des fonctions dans certaines ...

```
const myArray: number[ ] = [1, 2, 3, 4, 5];
myArray.sort((a, b) => a − b );

const myArray2: string[ ] = ['a', 'b', 'c', 'd', 'e'];
myArray2.sort((a, b) => a.localeCompare(b));

const myArray3: {id: number, name: string}[ ] = [
  {id: 1, name: 'alice'},
  {id: 2, name: 'bob'},
  {id: 3, name: 'charlie'},
];
myArray3.sort((a, b) => a.name.localeCompare(b.name));
```

# TypeScript : les interfaces



Les interfaces permettent de définir des types complexes.

```
interface Person {
  id: number;
  name: string;
}
const person: Person = {id: 1, name: 'alice'};
```